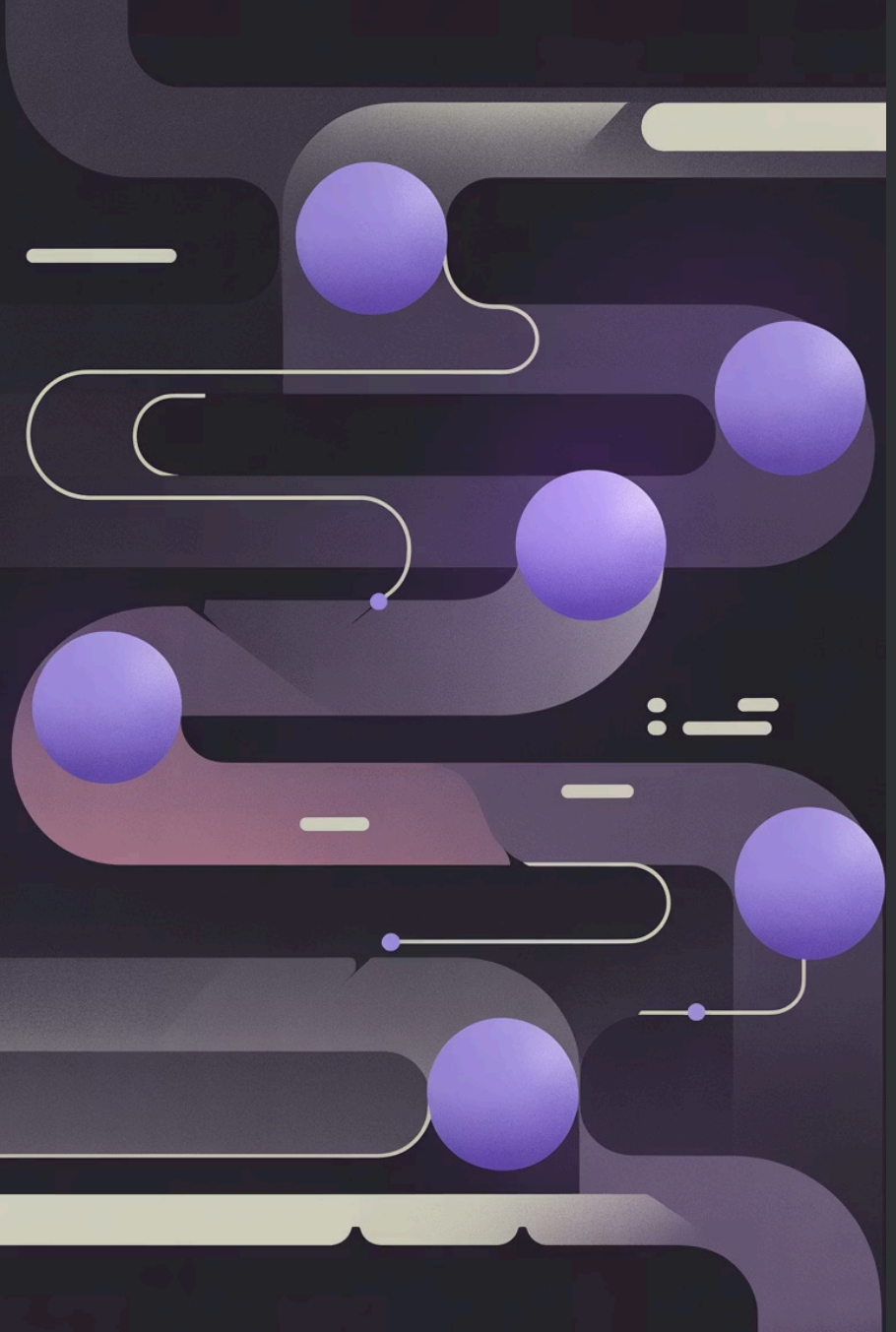

Results and Reflections

A technical summary of Pillar 1 of the Failure Scenario Generation System: architecture decisions, MVP results, deferred scope, and open questions.

Made with GAMMA

ARCHITECTURE

What Was Built

A multi-stage CLI pipeline: **GitHub API → parse → normalise → deduplicate → quality score → store**, with a separate **enrich** command for LLM classification.

Two-Model Data Flow

RawIncident → **IncidentRecord** (Pydantic). Invalid records logged and skipped. Garbage content never reaches storage.

Separate Enrichment Command

LLM enrichment decoupled from ingestion. Runs incrementally or with `--force`; failures don't block new records.

LangChain Abstraction

`.with_structured_output()` enforces typed Pydantic responses. Model swappable via config.

Config-Driven Taxonomy

Three-level taxonomy in `config.yaml`, injected at call time. Editable without code changes, enabling prompt caching.

Git-Object-Style Storage

JSON files in `output/incidents/ab/` subdirectories by ID prefix. Atomic writes via `tempfile + os.replace`.

SHA-Based Change Detection

Previous SHA cached in `run_state.json`. Unchanged source = skip crawl entirely, re-runs under one second.

Made with GAMMA

MVP RESULTS

Pipeline Output: danluu/post-mortems

~150

Incidents Parsed

Raw records from the README

~120

Eligible for Enrichment

After filtering invalid URLs and low-confidence parses

<5s

Full Run Time

Complete pipeline without enrichment

~$1.50

Estimated Corpus Cost

Full enrichment run, 15-20 min wall time

Before Enrichment

Quality scores cluster **0.3–0.6** — terse index entries, not detailed reports.

After Enrichment

Each record gains a structured LLM summary, refined root causes, severity classification, duration, and a three-level taxonomy label with justification. Scores shift upward as completeness and specificity improve.

Made with GAMMA

DEFERRED SCOPE

What Was Not Built

Intentionally Deferred

- **Automatic enrichment**: because the application is local, money is saved by only calling the enrich command when it is needed and on a select number of incidents
- **Concurrent enrichment**: sequential by design; thread pool deferred pending sequential validation
- **Multiple data sources**: parser tightly coupled to danluu README format
- **Database backend**: flat JSON sufficient at 150 records; pgsql planned for scale
- **API layer**: no REST/GraphQL; downstream reads JSON directly
- **Automated scheduling**: manually triggered; no cron or webhook integration

Harder Than Expected

- **Markdown parsing**: inconsistent entry formats required a state machine and careful fixture work
- **HTML extraction**: post-mortem pages vary enormously; truncation partially mitigates noise

Next With Another Week

1. Concurrent enrichment
2. Prompt caching to reduce toke cost
3. Second data source for schema validation
4. Evaluation harness with 50–100 hand-labelled records

Made with GAMMA

OPEN QUESTIONS

Unresolved Risks

LLM Extraction Accuracy

Schema conformance enforced, but factual accuracy unmeasured. No ground-truth benchmark exists.

Parse Confidence Calibration

Heuristic score unvalidated against human judgement; poor calibration degrades the enrichment corpus directly.

Taxonomy Coverage

Defined top-down; real incidents often span categories or fall outside represented types.

Dead Source URLs

Older links unreachable — systematic under-enrichment with no fallback or review flagging.

Quality Score Validity

Completeness and specificity are proxies for usefulness; correlation with downstream performance unknown.

Deduplication Threshold

SimHash at 0.9 set without empirical validation; optimal value likely varies by README section.

Made with GAMMA